

Programmed I/O

✓ It is one of the simplest forms of I/O where the CPU has to do all the work

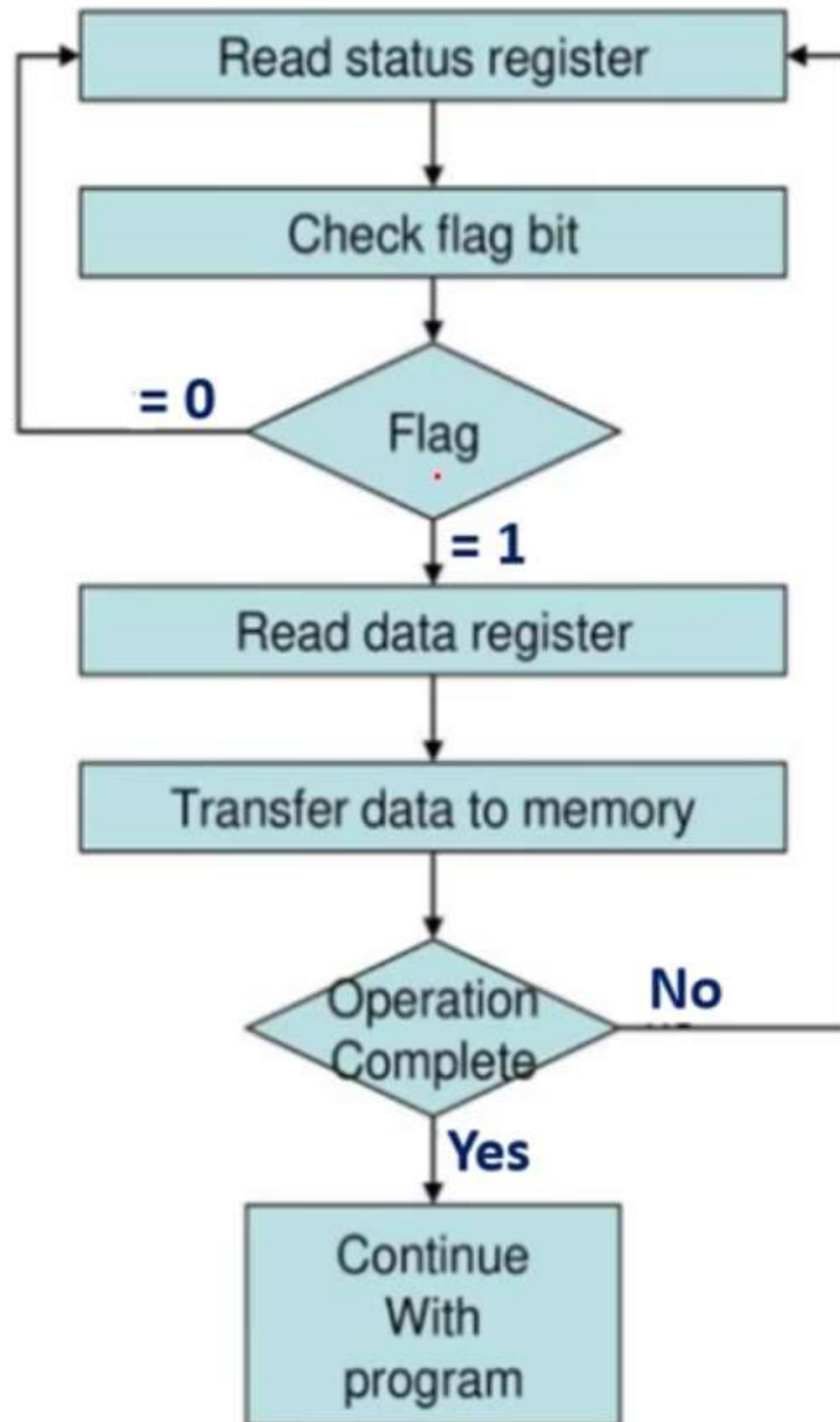
✓ Programmed I/O is a technique or approach that we use to transfer data between the processor and the I/O module

- ✓ ☐ It is **due to the result** of the I/O instructions that are written in the computer program
- ✓ ☐ Each data **item transfer** is **initiated** by an **instruction** in the program
- ✓ ☐ **Usually the transfer** is from a CPU register and memory
- ✓ ☐ In this case it **requires** constant **monitoring** by the CPU of the peripheral devices by using interface

Advantages & Disadvantages

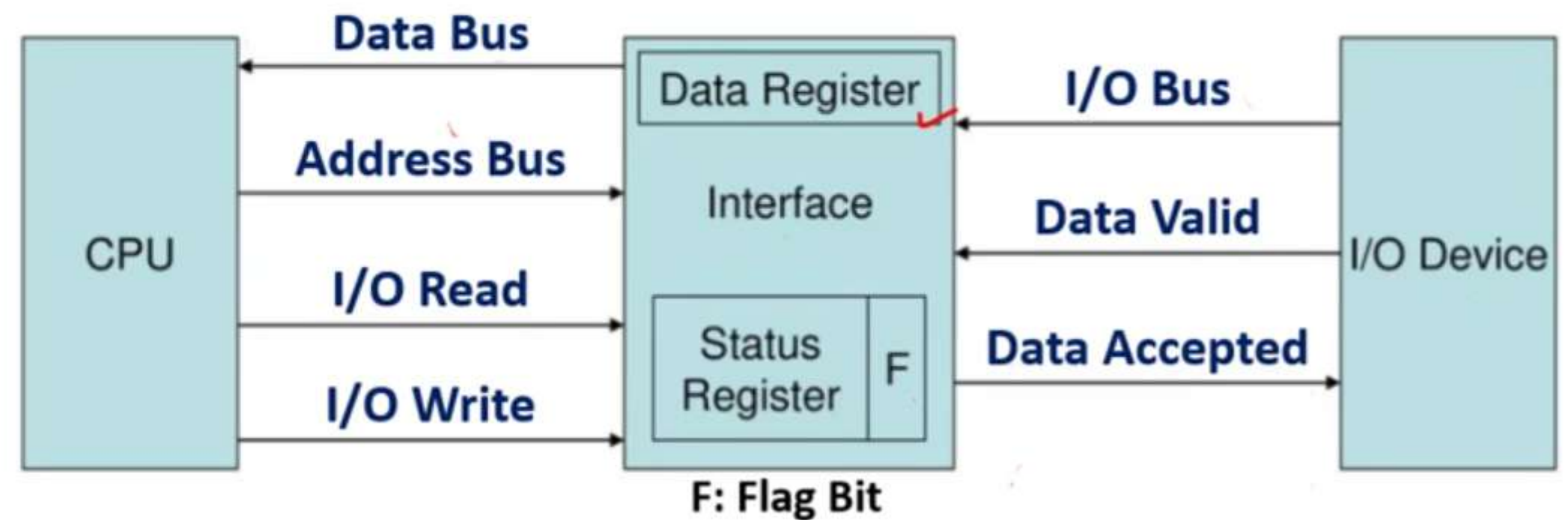
Advantages	▪ Simple to implement
	▪ Very little hardware support
Disadvantages	▪ Busy waiting
	▪ Ties up CPU for long period with no useful work

Programmed I/O



Flowchart for CPU Program to Input Data

- When a byte of data is available, the device places it in the I/O bus and enables its data valid line.
- The interface accepts the byte into its data register and enables the data accepted line.



Data Transfer from I/O Device to CPU

- The interface sets a bit in status register, referred as **F** or "**flag**" bit.
- The device can now disable the data valid line, but it will not transfer another byte until the data accepted line is disabled by the interface

Programmed I/O

- **In the programmed I/O method**, the I/O device does not have direct access to memory.
- **A transfer from an I/O device** to memory requires the execution of several instructions by the CPU, including an input instruction to transfer the data from the device to the CPU and a store instruction to transfer the data from the CPU to memory.
- **Other instructions** may be needed to verify that the data are available from the device and to count the numbers of words transferred.
- **An example of data transfer** from an I/O device through an interface into the CPU is shown in Fig. The device transfers bytes of data one at a time as they are available.
- **When a byte of data** is available, the device places it in the I/O bus and enables its data valid line.
- **The interface accepts** the byte into its data register and enables the data accepted line.
- **The interface sets** a bit in the status register that we will refer to as an F or "flag" bit. The device can now disable the data valid line, but it will not transfer another byte until the data accepted line is disabled by the interface.
- **A program is written** for the computer to check the flag in the status register to determine if a byte has been placed in the data register by the I/O device.
- **This is done by reading** the status register into a CPU register and checking the value of the flag bit.
- **If the flag is equal** to 1, the CPU reads the data from the data register.
- **The flag bit** is then cleared to 0 by either the CPU or the interface, depending on how the interface circuits are designed.
- **Once the flag is cleared**, the interface disables the data accepted line and the device can then transfer the next data byte.

Interrupt Initiated I/O

- ✓ ☐ An **alternative** to the CPU constantly monitoring the flag is to let the interface inform the computer when it is ready to transfer data
- ✓ ☐ This **mode of transfer** uses the **interrupt facility**. While the CPU is running a program, it does not check the flag.
- ✓ ☐ **However**, when the flag is set, the computer is momentarily interrupted from proceeding with the current program and is informed of the fact that the flag has been set
- ✓ ☐ The **CPU deviates** from what it is doing to take care of the input or output transfer
- ✓ ☐ **After the transfer** is completed, the computer returns to the previous program to continue what it was doing before the interrupt
- ✓ ☐ The **CPU responds** to the interrupt signal by storing the return address from the program counter into a memory stack and then control branches to a service routine that processes the required I/O transfer
- ✓ ☐ The **way that the processor chooses** the branch address of the service routine varies from one unit to another

In principle, there are two methods for accomplishing this:

- ☐ One is called **vectored interrupt** and the other, **nonvectored interrupt**
- ☐ In a **non vectored interrupt**, the branch address is assigned to a fixed location in memory.
- ☐ In a **vectored interrupt**, the source that interrupts supplies the branch information to the computer. This information is called the interrupt vector.
- ☐ In some computers the interrupt vector is the first address of the I/O service routine. In other computers the interrupt vector is an address that points to a location in memory where the beginning address of the I/O service routine is stored.

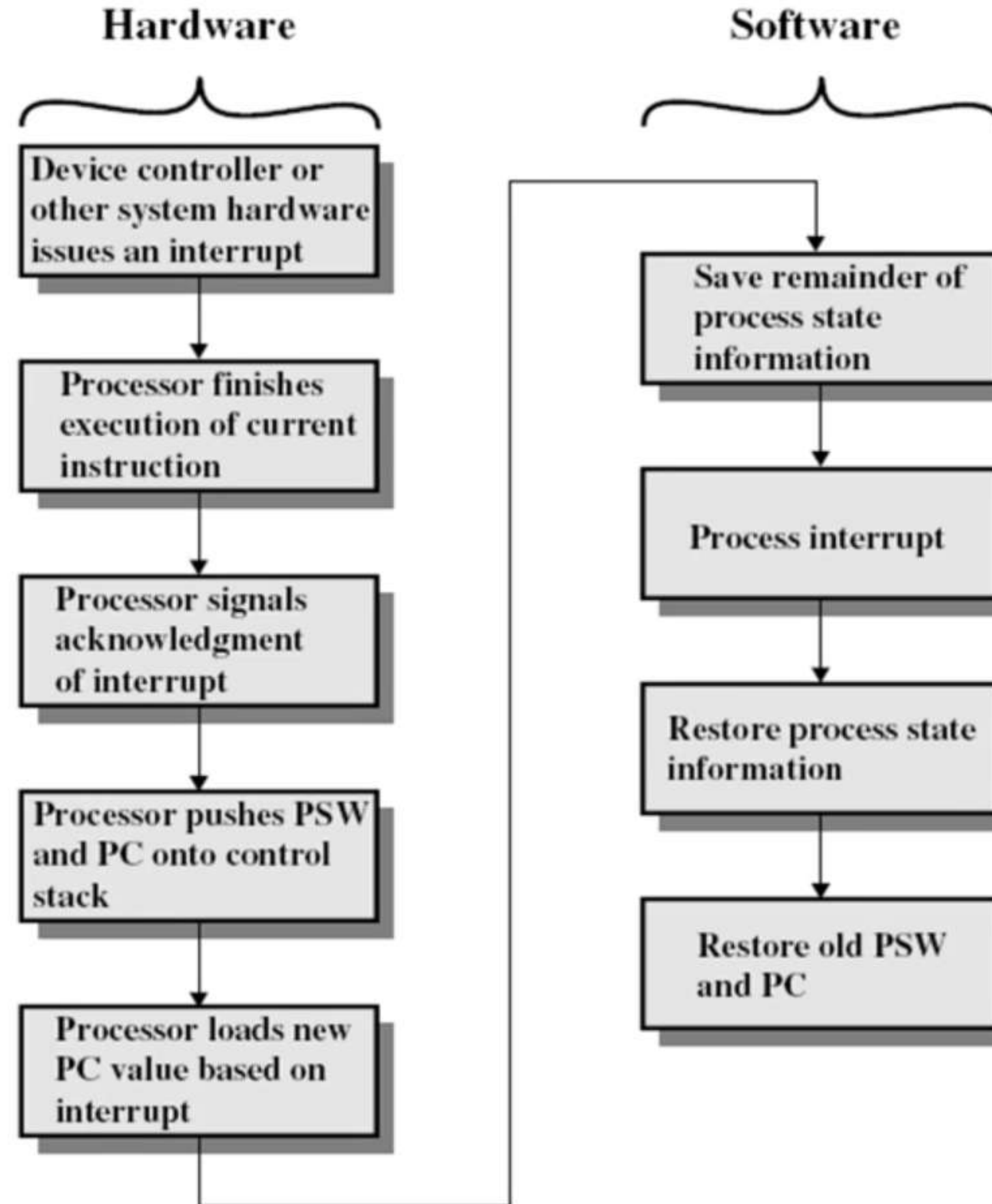
Advantages & Disadvantages

Advantages	▪ Fast
	▪ Efficient
Disadvantages	▪ Can be tough to get various pieces to work well together
	▪ Usually done by the hardware manufacturer / OS maker, e.g. Microsoft

Interrupt Initiated I/O

- ✓ The interrupt technique requires more **complex hardware and software**, but makes far more efficient use of the computer's time and capacities
- Figure shows the simple interrupt processing

- ❖ **For input**, the device interrupts the CPU when new data has arrived and is ready to be retrieved by the system processor. The actual actions to perform depend on whether the device uses *I/O ports* or *memory mapping*.
- ❖ **For output**, the device delivers an interrupt either when it is ready to accept new data or to acknowledge a successful data transfer. **Memory-mapped** and **DMA**-capable devices usually generate interrupts to tell the system they are done with the buffer.



**CPU
Viewpoint**

Interrupt Initiated I/O

Priority Interrupt

- Determines which **interrupt is to be served first** when **two or more requests** are made simultaneously
- Also determines which interrupts are permitted to interrupt the computer while another is being serviced
- Higher priority interrupts can make requests while servicing a lower priority interrupt

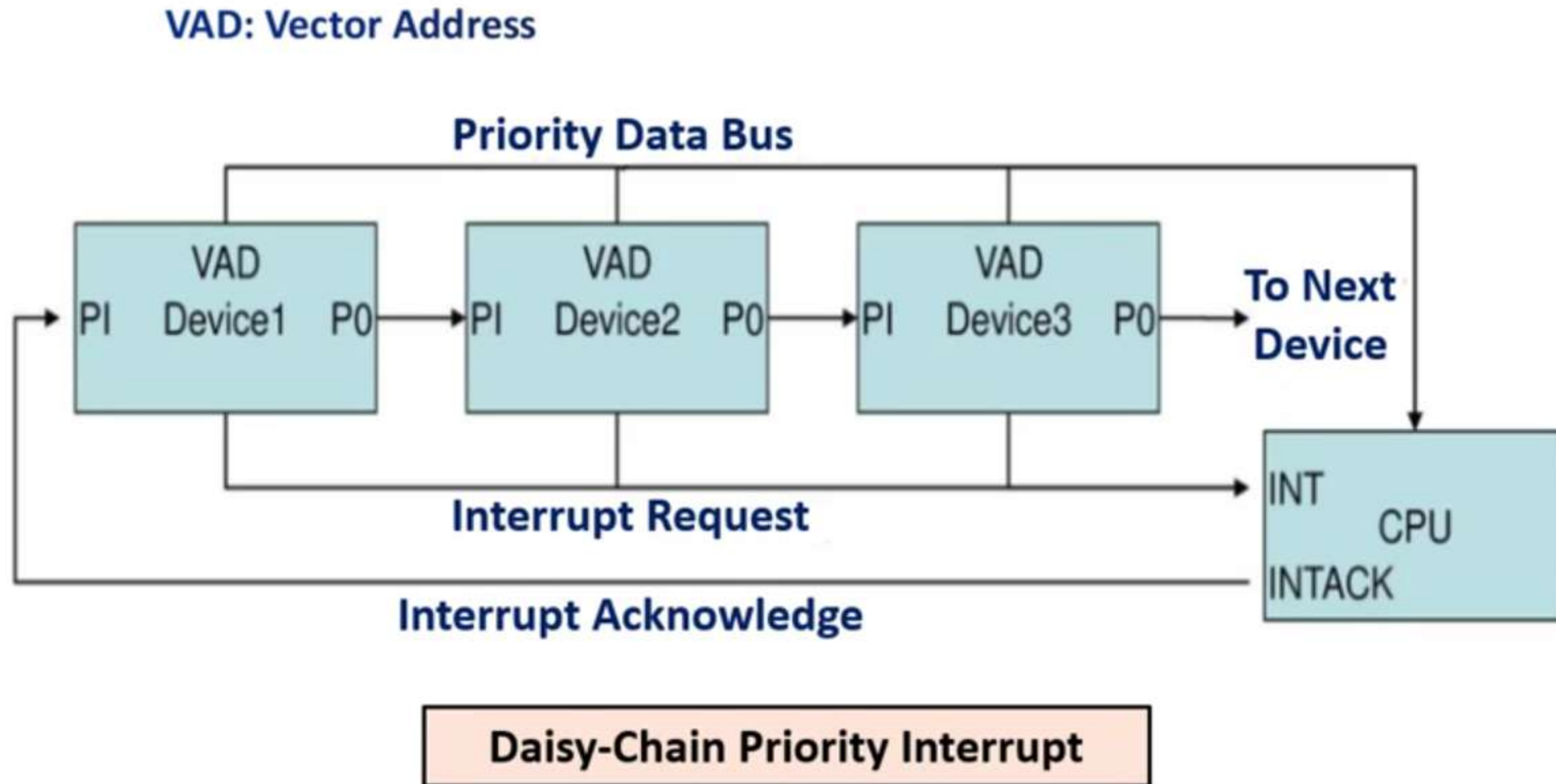
Implementing Priority Interrupt by Software (Polling)

- **Priority is established** by the **order of polling the devices** (interrupt sources), that is identify the highest-priority source by software means
- One common branch address is used for all interrupts
- **Program polls** the interrupt sources in sequence
- The highest-priority source is tested first
- Flexible since it is established by software
- Very slow & Low cost since it needs a very little hardware

Implementing Priority Interrupt by Hardware

- **Require a priority interrupt manager** which accepts all the interrupt requests to determine the highest priority request
- Fast since identification of the highest priority interrupt request is identified by the hardware
- Fast since each interrupt source has its own interrupt vector to access directly to its own service routine

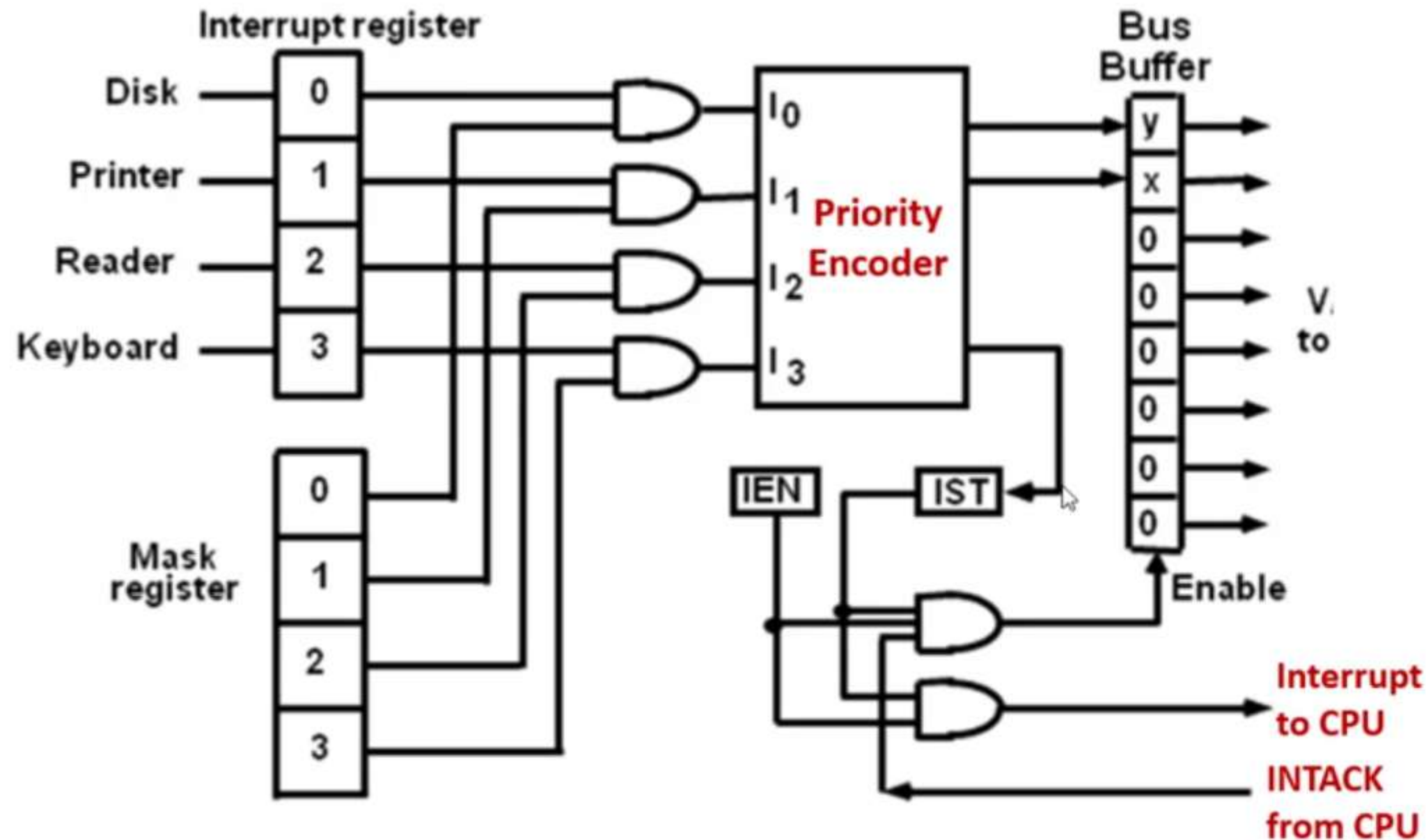
Daisy Chain Priority



This approach is similar to Polling but its hardware approach

- ❖ Interrupt Request from any device
- ❖ CPU responds by INTACK
- ❖ Any device receives signal(INTACK) at PI puts the VAD on the bus
- ❖ Among interrupt requesting devices the only device which is physically closest to CPU gets INTACK and it blocks INTACK to propagate to the next device

Parallel Priority Interrupt



Priority Encoder Truth Table

Inputs				Outputs			Boolean functions
I_0	I_1	I_2	I_3	x	y	IST	
1	d	d	d	0	0	1	$x = I_0' I_1'$ $y = I_0' I_1 + I_0' I_2'$ $(IST) = I_0 + I_1 + I_2 + I_3$
0	1	d	d	0	1	1	
0	0	1	d	1	0	1	
0	0	0	1	1	1	1	
0	0	0	0	d	d	0	

- **IEN:** Set or Clear by instructions ION or IOF
- **IST:** Represents an unmasked interrupt has INTACK enables tristate Bus Buffer to load VAD generated by the Priority Logic
- **Interrupt Register:**
 - ❖ Each bit is associated with an Interrupt Request from different Interrupt Source - different priority level
 - ❖ Each bit can be cleared by a program instruction
- **Mask Register:**
 - ❖ Mask Register is associated with Interrupt Register
 - ❖ Each bit can be set or cleared by an Instruction